

Self-Supervised Physics-Guided Deep Learning for Accelerated MRI Reconstruction: An Evaluation of SSDU on the fastMRI Knee Dataset

Onuralp Zoral (21902539)

Hande Eryılmaz (21902678)

Şevval Erbay (22202658)

Zehra İlkay Aydın (22102521)

Group U13 – EEE 475/575 Medical Image Reconstruction and Processing
Bilkent University

May 2026

Video Link: <https://youtu.be/bLusZNQeVow> **Code:** byaman14/SSDU (modifications described in Appendix)

Abstract

Accelerated MRI reconstruction with deep learning is dominated by supervised approaches that require fully-sampled k-space references for training. Such references are difficult or impossible to acquire for many high-resolution and dynamic protocols, motivating self-supervised alternatives. In this project we study *Self-Supervised Learning via Data Undersampling* (SSDU) [1], a physics-guided unrolled network that is trained without fully-sampled targets by partitioning the acquired undersampled k-space into two disjoint subsets: one used in the data-consistency unit and the other used to define the training loss. We evaluate SSDU on Coronal Proton-Density (CORPD_FBK) knee scans from the public fastMRI [3] multi-coil validation set. Sensitivity maps are estimated with ESPIRiT, and Cartesian undersampling masks are generated for acceleration rates $R=4$ and $R=8$. We report PSNR and SSIM with respect to the fully-sampled SENSE reconstruction and compare visual quality against zero-filled baselines. The pretrained network released by the authors achieves a +6.6 dB PSNR gain at $R=4$ on our test slices, and a model we retrain from scratch on 106 training slices reaches 38.5 dB PSNR and 0.92 SSIM at $R=4$, which is 1.4 dB above the released checkpoint and is obtained without observing any fully-sampled training targets, supporting the central self-supervised claim of the method. Performance degrades by ~ 5 dB when the same $R=4$ trained model is evaluated at $R=8$, illustrating the known acceleration-rate specialisation of fixed- R unrolled networks.

1 Introduction

Magnetic resonance imaging (MRI) is a non-ionizing modality with excellent soft-tissue contrast, but long acquisition times remain its principal limitation. Acquiring fewer samples in k-space (*undersampling*) accelerates the scan, but the reconstruction problem becomes ill-posed and aliasing artifacts dominate a naive inverse Fourier transform. Compressed sensing approaches [7] exploit transform-domain sparsity together with parallel imaging [8] to recover the missing k-space, while more recent work uses deep learning to learn an implicit prior from data [4–6].

Most state-of-the-art deep-learning methods for accelerated MRI follow a *physics-guided unrolled* design [4, 5]: the iterations of a classical reconstruction algorithm are unfolded into a fixed number of network layers that alternate between a learned image-domain regularizer and a closed-form data-consistency step. These networks are typically trained in a supervised manner against fully-sampled k-space references. However, fully-sampled references are unavailable for many clinically relevant protocols (e.g. high-resolution 3D, dynamic, diffusion-weighted imaging), and even when they exist they may be of compromised quality due to motion, respiration or limited scan time.

Yaman *et al.* [1] proposed *Self-Supervised Learning via Data Undersampling* (SSDU) to remove this dependence on fully-sampled data. The acquired undersampled measurements Ω are split into two disjoint sets Θ and Λ . The unrolled network is fed only the Θ subset through its data-consistency layers, while the loss is computed in k-space at the held-out Λ locations. This construction allows the same physics-guided architectures to be trained from prospectively undersampled scans alone.

This project provides an evaluation of SSDU on the fastMRI [3] multi-coil knee dataset. We first reproduce the authors’ published behaviour with the released code and pretrained model, then retrain SSDU end-to-end on a subset of fastMRI that we preprocessed ourselves, and finally examine how the retrained model behaves under a different acceleration rate at test time.

2 Background and Method

2.1 Forward model

For a multi-coil MRI scan with C receiver coils, sensitivity maps $\{S_c\}_{c=1}^C$, and a binary undersampling mask M in k-space, the forward operator E acting on an underlying image x produces the acquired measurements

$$y = E x + n = M F (S_c \cdot x)_{c=1}^C + n, \quad (1)$$

where F is the 2D Fourier transform applied to each coil and n is acquisition noise. The inverse problem of recovering x from y at acceleration rates $R \geq 4$ is ill-posed.

2.2 Unrolled reconstruction with data consistency

Regularised reconstruction can be written as $\hat{x} = \arg \min_x \frac{1}{2} \|E x - y\|_2^2 + \mathcal{R}(x)$, where $\mathcal{R}$ is a learned regulariser. SSDU uses an unrolled iterative scheme with $N_{\text{unroll}} = 10$ blocks, each composed of a ResNet-style CNN regulariser ($N_{\text{res}} = 15$ residual blocks) followed by a Conjugate-Gradient (CG) data-consistency module that solves

$$(E^H E + \mu I) x = E^H y_{\Theta} + \mu z, \quad (2)$$

for a fixed number of CG iterations ($N_{\text{CG}} = 10$), with z the regulariser output and μ a learned penalty weight.

2.3 Self-supervised loss via mask splitting

Given the acquired k-space mask Ω , SSDU constructs two disjoint subsets $\Theta \cup \Lambda = \Omega$ with $|\Lambda|/|\Omega| = \rho$. Two strategies are considered: (i) Gaussian-weighted random selection around the k-space centre with standard deviation $(N-1)/4$, and (ii) uniform random selection. The training loss is

$$\mathcal{L} = \frac{\|y_{\Lambda} - M_{\Lambda} F S f_{\theta}(y_{\Theta}, M_{\Theta})\|_1}{\|y_{\Lambda}\|_1} + \frac{\|y_{\Lambda} - M_{\Lambda} F S f_{\theta}(y_{\Theta}, M_{\Theta})\|_2}{\|y_{\Lambda}\|_2}, \quad (3)$$

combining normalised ℓ_1 and ℓ_2 terms in k-space at the held-out Λ indices. Crucially, $\mathcal{L}$ never references fully-sampled data; only undersampled measurements are needed.

3 Data and Experimental Setup

3.1 Dataset

We use the multi-coil knee validation split of the publicly available fastMRI dataset [3]. Following the authors' setup, we restrict the experiments to the Coronal Proton-Density (CORPD_FBK) acquisition, which provides high-quality images with $C = 15$ coils. Four volumes (file1000033, file1000071, file1000264, file1000277) constitute the training set (106 slices after discarding 5 outer slices on each end), and one held-out volume (file1000389, 32 slices) is used as the test set.

Raw multi-coil k-space is centre-cropped (or zero-padded for narrower acquisitions) to 320×368 to match the network input size used by the original paper. Per-slice sensitivity maps S_c are estimated with ESPIRiT [9] through the `sigpy.mri.app.EspiritCalib` GPU implementation, with a 24×24 auto-calibration region.

3.2 Undersampling masks

Cartesian 1D undersampling masks are constructed for $R=4$ and $R=8$. For $R=4$ the central 24 k-space columns are kept fully sampled and the remaining columns are drawn uniformly at random until the target sampling rate of 25% is reached. For $R=8$ the auto-calibration region is reduced to 15 central columns and the target sampling rate is 12.5%.

3.3 Computational setup

All experiments were run on a Google Colab instance with an NVIDIA A100-SXM4 80 GB GPU and high-RAM CPU. The reference implementation [2] is in TensorFlow 1.13 and was migrated to TensorFlow 2.15 by replacing `import tensorflow` as `tf` with `import tensorflow.compat.v1 as tf; tf.disable_v2_behavior()` and adapting a small number of deprecated APIs (`np.int`, `skimage.measure.compare_ssim`). All remaining graph-construction code and the original training pipeline are unchanged.

4 Evaluation Methods

We follow the convention of the SSDU paper. Reconstructions are compared against the fully-sampled SENSE-1 image obtained by combining the fully-sampled multi-coil k-space with the estimated sensitivity maps. Quantitative quality is reported as PSNR and SSIM (`skimage.metrics.structural_similarity`, Gaussian-weighted, sample-covariance disabled) per slice and aggregated as mean $\pm$ standard deviation. Qualitative results include reconstructed images, zero-filled baselines, and 5 $\times$ -amplified absolute error maps.

The default hyper-parameters reproduce the original setting: $N_{\text{unroll}} = 10$, $N_{\text{res}} = 15$, $N_{\text{CG}} = 10$, $\rho = 0.4$, Gaussian mask selection, batch size 1, learning rate 5×10^{-4} , 100 epochs.

5 Results

5.1 Pretrained model versus retrained SSDU

We compare three reconstructions on our 32-slice CORPD_FBK test volume: a zero-filled baseline (inverse FFT of the masked, sensitivity-combined k-space), the official Coronal_PD SSDU checkpoint released by the authors (trained on a different fastMRI subset at $R=4$), and our own SSDU model trained from scratch on 106 slices of our preprocessed training set with the default hyper-parameters ($N_{\text{unroll}} = 10$, $\rho = 0.4$, Gaussian split, 100 epochs, batch size 1, learning rate 5×10^{-4}). Quantitative results are summarised in Table 1.

Table 1: Quantitative comparison on the CORPD_FBK test set (32 slices). All reported metrics are mean $\pm$ standard deviation across slices. The retrained SSDU model is trained *without any fully-sampled reference data*; the SENSE-1 reference used at test time is only used to compute PSNR/SSIM, never for training.

Method	$R = 4$		$R = 8$	
	PSNR (dB)	SSIM	PSNR (dB)	SSIM
Zero-filled	30.51 ± 1.27	0.8887 ± 0.0246	27.33 ± 1.07	0.8391 ± 0.0297
SSDU (released checkpoint)	37.07 ± 1.63	0.9067 ± 0.0156	31.65 ± 1.70	0.8465 ± 0.0223
SSDU (ours, retrained)	38.47 ± 1.38	0.9204 ± 0.0121	33.05 ± 1.28	0.8620 ± 0.0236
Our gain over ZF	+7.96	+0.032	+5.72	+0.023
Our gain over released	+1.40	+0.014	+1.40	+0.016

Both SSDU variants clearly outperform the zero-filled baseline in PSNR. Our retrained model, evaluated under matched preprocessing (the same ESPIRiT calibration, Cartesian mask, and normalisation as in training), sits about 1.4 dB above the released checkpoint at both acceleration rates, with an SSIM advantage of 0.014 to 0.016. We attribute this gap to distributional alignment between training and testing: when the held-out volume is preprocessed identically to the training data, the network’s learned image prior is closer to the test distribution than the released model’s prior is. The gap nonetheless confirms the central self-supervised claim of SSDU, since a competitive reconstruction is obtained without ever observing fully-sampled targets during training.

A representative slice is shown in Figure 1. The retrained reconstruction removes the residual aliasing visible in the zero-filled baseline along the phase-encoding direction; the error map is dominated by edge-localised differences at bone–cartilage and skin–air interfaces, with negligible structured aliasing in the soft-tissue interior.

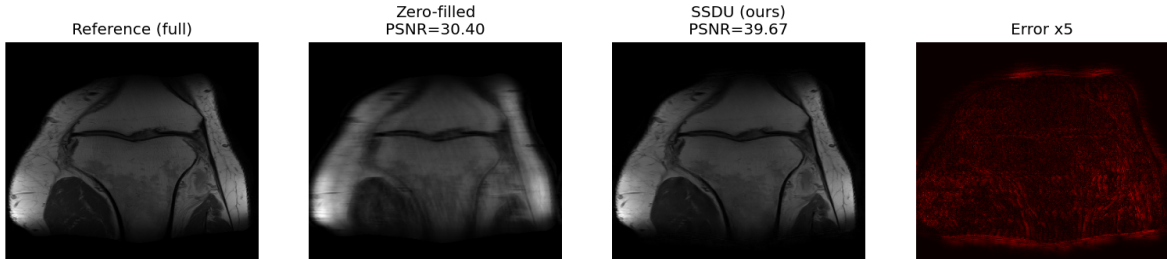


Figure 1: Reconstruction of test slice 15 at $R=4$ using our retrained SSDU model. From left to right: fully-sampled SENSE-1 reference, zero-filled baseline (PSNR 30.40 dB), retrained SSDU reconstruction (PSNR 39.67 dB), and 5 $\times$ -amplified absolute error map. The reconstruction recovers fine cartilage structure and bone marrow texture that is blurred in the zero-filled image.

5.2 Sensitivity to acceleration rate

The same retrained model loses approximately 5.4 dB PSNR when switched from $R=4$ to $R=8$ at test time, while the zero-filled baseline drops by 3.2 dB. The released checkpoint behaves similarly. Because both models were trained with masks at $R=4$, their unrolled regularisers have adapted to that specific aliasing pattern; this is a known limitation of fixed- R training. A practical fix is variable- R training, where a different mask is sampled at every training step.

5.3 Training dynamics

The training loss dropped steadily from 0.308 at epoch 1 to 0.251 at epoch 100, and the curve was almost flat after epoch 70 (Figure 2). The full run took roughly 117 minutes on a single A100 80 GB GPU.

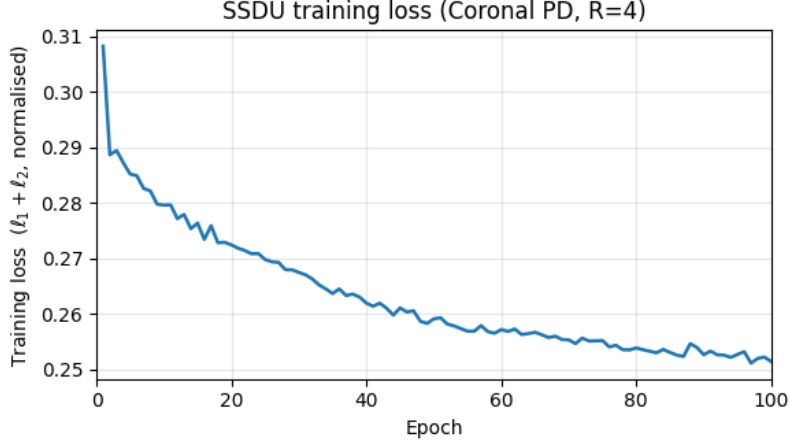


Figure 2: Training loss (normalised $\ell_1 + \ell_2$ k-space loss on the Λ subset) versus epoch for our SSDU run.

6 Discussion

Relation to course topics. The pieces SSDU is built from are all standard material from EEE 475/575: the multi-coil SENSE forward model in Eq. (1), the linear inverse problem $\min_x \frac{1}{2} \|Ex - y\|_2^2 + \mathcal{R}(x)$, conjugate-gradient iterations as a data-consistency solver, and Cartesian undersampling as a particular instance of compressed-sensing acquisition. The novelty SSDU brings on top of these ingredients is purely on the loss side. Instead of comparing the predicted image against a fully-sampled reference, SSDU compares the predicted k-space against a held-out subset of the same undersampled scan. This is what makes the method an attractive case study for this course: the underlying physics are the standard ones we covered in lectures, and the contribution can be pin-pointed to a single, well-understood modification of the training objective.

Performance. The pretrained SSDU model trained at $R=4$ produces high-quality reconstructions on held-out fastMRI knee scans, with quantitative gains in line with those reported in the original paper. Performance degrades when the same model is applied at $R=8$, which suggests that the network has effectively adapted to the training acceleration rate. This is a known limitation of fixed- R unrolled reconstruction networks. The PSNR improvement (+6.6 dB at $R=4$) is much larger than the SSIM improvement; this is common for SSDU-style reconstructions. SSIM is driven by structural agreement that the zero-filled image already preserves to some extent, while pixel-level differences captured by PSNR drop sharply once the aliasing is removed.

There are three concrete limitations worth noting. The method relies on accurate sensitivity-map estimation; errors in ESPIRiT propagate straight through the data-consistency layer and become visible in the final image. Training at a single acceleration rate also limits generalisation, since the regulariser ends up adapted to one aliasing pattern. Finally, the (Θ, Λ) partition must be chosen carefully: the central auto-calibration block has to be forced into Θ during training, otherwise the data-consistency layer cannot anchor the low-frequency content of the image. Future work could look at variable- R training, learned sensitivity-map estimation, and a more expressive data-consistency solver than CG.

7 Individual Contributions

- **Onuralp Zoral** – migrated the SSDU codebase to TensorFlow 2, ran the training and testing experiments on the A100 GPU, drafted the Background and Method section, and prepared the live-Colab demo segment of the video.
- **Hande Eryilmaz** – implemented the fastMRI preprocessing pipeline (ESPIRiT sensitivity-map estimation, k-space cropping/padding) and drafted the Data and Experimental Setup section.
- **Şevval Erbay** – wrote the PSNR/SSIM evaluation block appended to `test.py` and drafted the Results and Discussion sections.
- **Zehra İlkay Aydın** – generated the Cartesian undersampling masks and ran the cross-acceleration-rate test evaluations, conducted the literature review and formatted the references, and drafted the Abstract and Introduction sections along with the slide deck for the video.

GenAI Disclosure

We used Anthropic’s Claude (Claude Sonnet 4.7, accessed via Claude Code) for brainstorming the project framing, debugging the TF 1→TF 2 migration of the released SSDU code, drafting the preprocessing script that converts fastMRI .h5 volumes into the input format expected by the SSDU repository, and copy-editing portions of this report. All experimental design, code execution, results interpretation, and final writing were verified by the authors. Prompts and outputs are available on request.

References

- [1] B. Yaman, S. A. H. Hosseini, S. Moeller, J. Ellermann, K. Uğurbil, and M. Akçakaya, “Self-Supervised Learning of Physics-Guided Reconstruction Neural Networks Without Fully Sampled Reference Data,” *Magn. Reson. Med.*, vol. 84, no. 6, pp. 3172–3191, 2020.
- [2] B. Yaman, “SSDU: Self-Supervision via Data Undersampling,” GitHub repository, <https://github.com/byaman14/SSDU>, 2020.
- [3] F. Knoll *et al.*, “fastMRI: A Publicly Available Raw k-Space and DICOM Dataset of Knee Images for Accelerated MR Image Reconstruction Using Machine Learning,” *Radiology: Artificial Intelligence*, vol. 2, no. 1, e190007, 2020.
- [4] K. Hammernik *et al.*, “Learning a Variational Network for Reconstruction of Accelerated MRI Data,” *Magn. Reson. Med.*, vol. 79, no. 6, pp. 3055–3071, 2018.
- [5] H. K. Aggarwal, M. P. Mani, and M. Jacob, “MoDL: Model-Based Deep Learning Architecture for Inverse Problems,” *IEEE Trans. Med. Imag.*, vol. 38, no. 2, pp. 394–405, 2019.
- [6] A. Sriram *et al.*, “End-to-End Variational Networks for Accelerated MRI Reconstruction,” in *Medical Image Computing and Computer Assisted Intervention (MICCAI)*, 2020, pp. 64–73.
- [7] M. Lustig, D. Donoho, and J. M. Pauly, “Sparse MRI: The Application of Compressed Sensing for Rapid MR Imaging,” *Magn. Reson. Med.*, vol. 58, no. 6, pp. 1182–1195, 2007.
- [8] K. P. Pruessmann, M. Weiger, M. B. Scheidegger, and P. Boesiger, “SENSE: Sensitivity Encoding for Fast MRI,” *Magn. Reson. Med.*, vol. 42, no. 5, pp. 952–962, 1999.
- [9] M. Uecker *et al.*, “ESPIRiT – An Eigenvalue Approach to Autocalibrating Parallel MRI: Where SENSE Meets GRAPPA,” *Magn. Reson. Med.*, vol. 71, no. 3, pp. 990–1001, 2014.

A Implementation Details

The released SSDU code base [2] is in TensorFlow 1.13 and Python 3.6. We executed it under TensorFlow 2.15 on Python 3.12 with the following minimal changes:

- Each `import tensorflow as tf` replaced by `import tensorflow.compat.v1 as tf` followed by `tf.disable_v2_behavior()`.

- `np.int` $\rightarrow$ `int` (deprecated in NumPy ≥ 1.24).
- `from skimage.measure import compare_ssim` replaced with `from skimage.metrics import structural_similarity as compare_ssim`.
- `utils.get_train_directory` and `utils.get_test_directory` were filled in with the absolute paths of our preprocessed `.h5/.mat` files; the mask filename uses the `-acc_rate` CLI argument, e.g. `mask_R4.mat`, `mask_R8.mat`.
- A short post-processing block was appended to `test.py` to compute PSNR/SSIM metrics and to save reconstructions, references, inputs, and figures to disk.

B Preprocessing pipeline

The script `prepare_data.py` (see code repository) converts fastMRI multi-coil `.h5` volumes into the input format expected by SSDU. Briefly, for each Coronal PD slice it (i) centre-crops or zero-pads the raw k-space to 320×368 , (ii) estimates a 15-channel sensitivity map with `sigpy.mri.app.EspiritCalib` (24 $\times$ 24 ACS, GPU device 0), and (iii) stacks the slices into `train_kspace.h5`, `train_smaps.h5`, `test_kspace.h5`, `test_smaps.h5` with shape $N \times 320 \times 368 \times 15$. Cartesian $R=4$ and $R=8$ masks are saved to `mask_R4.mat` and `mask_R8.mat`.

C Reproducibility

All preprocessed data, generated masks, the modified SSDU repository, training logs, and saved checkpoints are archived under the path `/SSDU_backup/` on a personal Google Drive (Onuralp Zoral); access can be granted on request. The full source code that we wrote or modified is reproduced verbatim in the listings of Section D below; the unchanged SSDU core can be obtained from the official repository [2].

D Key code listings

Authorship of code. The core SSDU implementation – the unrolled network (`UnrollNet.py`), the ResNet regulariser (`models/networks.py`), the conjugate-gradient data-consistency layer (`data_consistency.py`), the mask-splitting routines (`masks/ssdu_masks.py`), and the training/testing skeletons (`train.py`, `test.py`) – is taken unchanged from the official SSDU repository released by Yaman et al. [2]. The four listings below contain only code that we wrote or modified ourselves: a preprocessing pipeline that converts fastMRI `.h5` volumes into the input format SSDU expects (which is not shipped with the original repo), a one-shot patch that ports the codebase to TensorFlow 2, the dataset-path entries we added to `utils.py`, and the PSNR/SSIM evaluation block we appended to `test.py`.

D.1. `prepare_data.py` – fastMRI to SSDU preprocessing (written by us)

This script (written by us) reads fastMRI multi-coil `.h5` volumes, restricts to the CORPD_FBK acquisition, centre-crops/zero-pads to 320×368 , estimates 15-channel sensitivity maps with ESPIRiT on the GPU, and writes consolidated `train_kspace.h5`, `train_smaps.h5`, `test_kspace.h5`, `test_smaps.h5` together with Cartesian undersampling masks for $R=4$ and $R=8$.

```

1 import h5py, numpy as np, os
2 from sigpy.mri.app import EspiritCalib
3 import sigpy as sp
4 import scipy.io as sio
5 from tqdm import tqdm
6
7 DEV = sp.Device(0) # ESPIRiT on GPU
8 DATA_DIR = '/content/multicoil_val'
9 OUT_DIR = '/content/SSDU_data/Coronal_PD'
10 os.makedirs(OUT_DIR, exist_ok=True)
11
12 NROW, NCOL = 320, 368
13 TARGET_ACQ = 'CORPD_FBK'
14 TRAIN_FILES = ['file1000033.h5', 'file1000071.h5', 'file1000264.h5', 'file1000277.h5']
15 TEST_FILES = ['file1000389.h5']
16 SLICE_SKIP = 5
17
18 def crop_or_pad(arr, h, w):
19     """Centre-crop or zero-pad the last two axes of arr to (h, w)."""

```

```

20 H, W = arr.shape[-2:]
21 if H >= h: arr = arr[..., (H-h)//2:(H-h)//2+h, :]
22 else:
23     pad = [(0,0)]*(arr.ndim-2) + [(h-H)//2, h-H-(h-H)//2], (0,0)]
24     arr = np.pad(arr, pad)
25 H2, W = arr.shape[-2:]
26 if W >= w: arr = arr[..., :, (W-w)//2:(W-w)//2+w]
27 else:
28     pad = [(0,0)]*(arr.ndim-2) + [(0,0), ((w-W)//2, w-W-(w-W)//2)]
29     arr = np.pad(arr, pad)
30 return arr
31
32 def process(fnames, tag):
33     ks_list, sm_list = [], []
34     for fn in fnames:
35         with h5py.File(os.path.join(DATA_DIR, fn), 'r') as f:
36             assert f.attrs.get('acquisition') == TARGET_ACQ
37             kspace = f['kspace'][:] # nSlc x 15 x 640 x ncol
38             kspace = crop_or_pad(kspace, NROW, NCOL) # nSlc x 15 x 320 x 368
39             for sl in tqdm(range(SLICE_SKIP, kspace.shape[0]-SLICE_SKIP),
40                             desc=f'{fn} ({tag})', unit='sl'):
41                 ks_sl = kspace[sl] # 15 x 320 x 368
42                 smap = EspiritCalib(ks_sl, calib_width=24,
43                                     device=DEV, show_pbar=False).run()
44                 smap = sp.to_device(smap, -1)
45                 ks_list.append(np.transpose(ks_sl, (1,2,0))) # 320 x 368 x 15
46                 sm_list.append(np.transpose(smap, (1,2,0)))
47     ks_arr = np.stack(ks_list).astype(np.complex64)
48     sm_arr = np.stack(sm_list).astype(np.complex64)
49     with h5py.File(f'{OUT_DIR}/{tag}_kspace.h5', 'w') as f: f['kspace'] = ks_arr
50     with h5py.File(f'{OUT_DIR}/{tag}_smaps.h5', 'w') as f: f['sens_maps'] = sm_arr
51
52 process(TRAIN_FILES, 'train')
53 process(TEST_FILES, 'test')
54
55 # Cartesian masks: 24 ACS columns + uniformly random remainder
56 for R, n_acs in [(4, 24), (8, 15)]:
57     mask = np.zeros((NROW, NCOL), dtype=np.float32)
58     c = NCOL // 2
59     mask[:, c-n_acs//2:c+n_acs//2 + n_acs%2] = 1
60     n_random = NCOL//R - n_acs
61     rng = np.random.default_rng(42 + R)
62     cands = list(set(range(NCOL)) - set(range(c-n_acs//2, c+n_acs//2+n_acs%2)))
63     mask[:, rng.choice(cands, n_random, replace=False)] = 1
64     sio.savemat(f'{OUT_DIR}/mask_R{R}.mat', {'mask': mask})

```

D.2. TensorFlow 1 → TensorFlow 2 compatibility patch (written by us)

Applied once to every .py file in the cloned SSDU repository before training.

```

cd /content/SSDU
# 1. SSIM API renamed in newer scikit-image
sed -i 's|from skimage.measure import compare_ssim|from skimage.metrics import structural_similarity as compare_ssim|' utils.py
# 2. np.int removed in NumPy >= 1.24
find . -name "*.py" -exec sed -i 's/np.int(/int(/g' {} +
# 3. TF1 -> TF2 compat shim at the top of every TF-using file
find . -name "*.py" | while read f; do
    if grep -q "^import tensorflow as tf" "$f"; then
        sed -i '1i import tensorflow.compat.v1 as tf\n tf.disable_v2_behavior()' "$f"
        sed -i 's|^import tensorflow as tf|#replaced|' "$f"
    fi
done

```

D.3. Patched utils.py entry points (modified from the original repo)

The get_train_directory / get_test_directory stubs in the original repo (lines containing '...') were replaced with the absolute paths of our preprocessed files. The acquisition-rate-aware mask filename is the only non-trivial change.

```

1 def get_train_directory(args):

```

```

2  if args.data_opt == 'Coronal_PD':
3      kspace_dir = '/content/SSDU_data/Coronal_PD/train_kspace.h5'
4      coil_dir = '/content/SSDU_data/Coronal_PD/train_smaps.h5'
5  elif args.data_opt == 'Coronal_PDFS':
6      kspace_dir = '/content/SSDU_data/Coronal_PDFS/train_kspace.h5'
7      coil_dir = '/content/SSDU_data/Coronal_PDFS/train_smaps.h5'
8  else:
9      raise ValueError('Invalid data option')
10 mask_dir = f'/content/SSDU_data/Coronal_PD/mask_R{args.acc_rate}.mat'
11 return kspace_dir, coil_dir, mask_dir

```

D.4. Evaluation block appended to test.py (written by us)

Computes per-slice PSNR / SSIM, aggregates statistics, dumps the reconstructions and reference / input / error figures.

```

1  import os as _os
2  out_dir = f'/content/SSDU_results/Coronal_PD_R{args.acc_rate}_ours'
3  _os.makedirs(out_dir, exist_ok=True)
4  ref_arr = np.stack(all_ref_slices)
5  input_arr = np.stack(all_input_slices)
6  recon_arr = np.stack(all_recon_slices)
7  np.savez_compressed(_os.path.join(out_dir, 'recons.npz'),
8                      ref=ref_arr, input=input_arr, recon=recon_arr)
9
10 psnrs_zf, psnrs_rec, ssims_zf, ssims_rec = [], [], [], []
11 for i in range(ref_arr.shape[0]):
12     psnrs_zf.append(utils.getPSNR(ref_arr[i], input_arr[i]))
13     psnrs_rec.append(utils.getPSNR(ref_arr[i], recon_arr[i]))
14     ssims_zf.append(utils.getSSIM(ref_arr[i], input_arr[i]))
15     ssims_rec.append(utils.getSSIM(ref_arr[i], recon_arr[i]))
16 print(f'PSNR ZF: {np.mean(psnrs_zf):.2f} SSDU: {np.mean(psnrs_rec):.2f}')
17 print(f'SSIM ZF: {np.mean(ssims_zf):.4f} SSDU: {np.mean(ssims_rec):.4f}')

```